

TOUCHE PAS AU KLAXON

— Application de covoiturage inter - sites —

Dossier de projet – Examen TP DWWM

Mai 2026

Johanne Suire

Formation BTS / DWWM

Architecture MVC – PHP / MySQL / Bootstrap

GitHub - MCD - MLD - Installation - Sécurité - Architecture

SOMMAIRE

1. Présentation du projet.....	3
2. Contexte et besoin.....	3
3. Objectifs du projet.....	4
4. Contraintes du cahier des charges.....	4
5. Description des fonctionnalités et des pages.....	5
5.1. Page d'accueil (visiteur non connecté).....	5
5.2. Authentification et espace utilisateur.....	5
5.3. Création et modification de trajet.....	5
5.4. Tableau de bord administrateur.....	5
6. Lien vers le dépôt GitHub.....	6
7. Modèle Conceptuel de Données (MCD).....	6
8. Modèle Logique de Données (MLD).....	7
9. Procédure d'installation.....	7
10. Identifiants de connexion.....	9
11. Technologies utilisées.....	9
12. Mesures de sécurité.....	10
13. Structure du code.....	11
14. Bilan personnel, difficultés rencontrées et axes d'amélioration.....	11

1. Présentation du projet

Le projet présenté s'intitule Touche pas au klaxon. Il s'agit d'une application web de covoiturage inter-sites réalisée dans le cadre d'un devoir de formation, à partir d'un brief imposé, avec un temps limité de 7 h 30.

Cette application a pour objectif de diffuser au sein d'une entreprise les trajets prévus entre plusieurs implantations géographiques afin de favoriser le covoiturage et de limiter les déplacements effectués avec des véhicules peu occupés. Le site est destiné à être utilisé sur l'intranet de l'entreprise.

Dans cette première version, l'application permet d'afficher les trajets disponibles, de gérer l'authentification des utilisateurs, de proposer un trajet, de modifier ses propres trajets et d'administrer les agences et les trajets via un espace réservé à l'administrateur.

Le projet a été développé en respectant les contraintes techniques du brief, notamment l'utilisation de PHP, d'une architecture MVC, d'une base de données MySQL ou MariaDB, ainsi que de Bootstrap et Sass pour l'interface. Le développement devait également intégrer une documentation du code, une analyse statique avec PHPStan et des tests unitaires avec PHPUnit.

2. Contexte et besoin

L'entreprise décrite dans le brief possède plusieurs sites géographiques entre lesquels les salariés se déplacent régulièrement pour des raisons professionnelles. Dans la situation actuelle, il arrive que plusieurs véhicules effectuent le même trajet le même jour, avec un faible taux d'occupation, ce qui génère des coûts inutiles et un impact environnemental négatif.

Pour répondre à ce problème, la direction souhaite mettre en place sur l'intranet une application de covoiturage interne. Cette application doit permettre de diffuser les trajets prévus entre les différents sites de l'entreprise afin de faciliter la mise en relation des collaborateurs qui effectuent les mêmes déplacements. L'objectif est de réduire le nombre de véhicules utilisés, d'optimiser le remplissage des trajets et de favoriser une démarche plus éco-responsable.

Le besoin fonctionnel de cette première version reste volontairement limité : une page d'accueil accessible à tous doit afficher les trajets futurs pour lesquels il reste des places disponibles, triés par date de départ. Après authentification, les employés doivent pouvoir consulter plus de détails sur un trajet, proposer leurs propres trajets et modifier ceux dont ils sont auteurs. Un administrateur dispose, quant à lui, de droits étendus pour gérer les agences, les trajets et les données affichées dans l'application.

3. Objectifs du projet

Du point de vue fonctionnel, l'objectif principal du projet est de fournir une application simple d'utilisation permettant aux salariés de visualiser les trajets inter-sites disponibles et de proposer leurs propres trajets de covoiturage. L'application doit permettre à un visiteur non connecté de consulter la liste des trajets à venir avec des places disponibles, puis, après authentification, d'accéder à des informations détaillées et à des actions de gestion sur ses trajets.

Plus précisément, le projet doit offrir trois niveaux d'utilisation : un accès public à la liste des trajets futurs avec au moins une place libre, un espace utilisateur permettant de consulter les coordonnées du conducteur, de créer un trajet et de modifier ou supprimer ses propres trajets, et enfin un tableau de bord administrateur donnant la possibilité de lister et gérer les utilisateurs, les agences et les trajets. Ces fonctionnalités doivent respecter les règles de cohérence métier définies dans le cahier des charges (dates cohérentes, agences de départ et d'arrivée distinctes, gestion des places disponibles).

Sur le plan technique, le projet vise également à mettre en œuvre une architecture PHP de type MVC reposant sur une base de données MySQL ou MariaDB, en s'appuyant sur Bootstrap et Sass pour l'interface. Il a aussi pour objectif de respecter des exigences de qualité de code et de sécurité : code documenté en DocBlock, analyse statique avec PHPStan, intégration de tests unitaires PHPUnit sur les opérations d'écriture en base de données, et mise en place de bonnes pratiques de sécurité adaptées à une application intranet.

4. Contraintes du cahier des charges

Le projet Touche pas au klaxon a été réalisé dans un cadre fortement contraint, défini par le brief de l'organisme de formation. Sur le plan technique, le site devait obligatoirement être développé en PHP en adoptant une architecture MVC et en s'appuyant sur une base de données relationnelle MySQL ou MariaDB. L'application devait être déployable sur l'intranet de l'entreprise et fonctionner dans un contexte multi-utilisateurs (visiteurs, salariés connectés, administrateur).

Au niveau de l'interface, l'identité graphique n'était pas l'objectif principal du devoir, mais l'utilisation de Bootstrap et de Sass était imposée, ainsi qu'une palette de couleurs précise à respecter via les variables de Bootstrap. Cette contrainte visait à préparer la réutilisation du code pour d'autres sites thématiques similaires, qui pourront changer de palette sans modifier la structure générale.

Le cahier des charges imposait également plusieurs exigences de qualité de code : les classes devaient être commentées avec DocBlock, le projet devait intégrer une analyse statique à l'aide de PHPStan et des tests unitaires PHPUnit couvrant au minimum les fonctionnalités réalisant des opérations d'écriture en base de données. Enfin, le travail devait être réalisé dans un temps limité de 7 h 30, ce qui constituait une contrainte supplémentaire sur la planification du développement et la priorisation des fonctionnalités.

5. Description des fonctionnalités et des pages

5.1. Page d'accueil (visiteur non connecté)

La page d'accueil est accessible sans authentification et présente la liste des trajets planifiés pour lesquels il reste au moins une place disponible. Les trajets passés sont filtrés et n'apparaissent pas, et la liste est triée par date et heure de départ croissantes afin de mettre en avant les prochains déplacements. Pour chaque trajet, la page affiche l'agence de départ, la date et l'heure de départ, l'agence d'arrivée, la date et l'heure d'arrivée, ainsi que le nombre de places encore disponibles.

5.2. Authentification et espace utilisateur

L'application propose un formulaire de connexion permettant à un salarié de s'authentifier à l'aide de son adresse email et de son mot de passe. Une fois connecté, l'utilisateur retrouve la liste des trajets disponibles, mais bénéficie de fonctionnalités supplémentaires : un bouton permet d'ouvrir une fenêtre modale affichant les informations détaillées d'un trajet (identité de la personne qui propose le trajet, numéro de téléphone, adresse email, nombre total de places). Si l'utilisateur est l'auteur d'un trajet, il dispose en plus de boutons pour modifier ou supprimer ce trajet.

5.3. Création et modification de trajet

Une page dédiée permet à un utilisateur connecté de créer un nouveau trajet. Les informations le concernant (nom, prénom, adresse email et téléphone) sont pré-remplies et non modifiables, car elles proviennent des données RH fournies en annexe du devoir. Le formulaire impose plusieurs contrôles de cohérence : agence de départ et agence d'arrivée différentes, impossibilité de renseigner une date d'arrivée antérieure à la date de départ, gestion du nombre total de places et des places disponibles avec des valeurs strictement positives et cohérentes. Une interface similaire est utilisée pour la modification des trajets dont l'utilisateur est l'auteur.

5.4. Tableau de bord administrateur

L'administrateur dispose d'un tableau de bord spécifique accessible après authentification avec un compte de rôle administrateur. Cette interface lui permet de lister les utilisateurs, les agences et les trajets enregistrés dans le système. Il peut créer, modifier et supprimer une agence, ce qui lui donne la maîtrise de la liste des villes disponibles pour les trajets. Il peut également lister et supprimer des trajets en cas d'erreur ou de contenu non conforme. Cet espace centralise ainsi les opérations de gestion nécessaires au bon fonctionnement de l'application.

6. Lien vers le dépôt GitHub

Le code source complet du projet est disponible sur GitHub :
<https://github.com/Maygae/touche-pas-au-klaxon.git>

Ce dépôt contient l'ensemble des éléments nécessaires à la compréhension, à l'installation et à l'exécution du projet :

- Le code source complet de l'application développé en PHP selon une architecture MVC.
- Le script de création de la base de données (`sql/schema.sql`).
- Le script d'alimentation de la base de données avec un jeu d'essais (`sql/seed.sql`).
- Un fichier `README.md` décrivant l'application, son installation et son utilisation.
- Les tests unitaires réalisés avec PHPUnit dans le dossier `tests/`.

7. Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données représente les entités et les relations nécessaires au fonctionnement de l'application de covoiturage inter-sites. Il permet de structurer les informations sur les utilisateurs, les trajets et les agences de manière cohérente par rapport au cahier des charges. Le schéma graphique complet est fourni dans le fichier séparé **MCD.pdf**.

Entités :

Entité	Description
UTILISATEUR	Employés de l'entreprise (données issues du système RH)
TRAJET	Trajets de covoiturage proposés entre les agences
AGENCE	Agences (villes) de l'entreprise

Relations :

Relation	Cardinalités	Description
PROPOSER	UTILISATEUR (1,N) → TRAJET (1,1)	Un utilisateur peut proposer un ou plusieurs trajets.
DÉPART DE	TRAJET (1,1) → AGENCE (0,N)	Chaque trajet part d'une agence.
ARRIVÉE À	TRAJET (1,1) → AGENCE (0,N)	Chaque trajet part d'une agence.

8. Modèle Logique de Données (MLD)

Le modèle logique de données décrit la structure détaillée de la base de données relationnelle utilisée par l'application. Il précise les tables, leurs attributs, les clés primaires et étrangères, ainsi que les principales contraintes d'intégrité.

Tables

- **agences**
(id INT, ville VARCHAR(100) UNIQUE NOT NULL, created_at DATETIME, updated_at DATETIME)
- **utilisateurs**
(id INT, nom VARCHAR(100) NOT NULL, prenom VARCHAR(100) NOT NULL, telephone VARCHAR(20) NOT NULL, email VARCHAR(255) UNIQUE NOT NULL, mot_de_passe VARCHAR(255) NOT NULL, role ENUM('utilisateur','admin') NOT NULL, created_at DATETIME, updated_at DATETIME)
- **trajets**
(id INT, #agence_depart_id INT REFERENCES agences(id), #agence_arrivee_id INT REFERENCES agences(id), date_depart DATETIME NOT NULL, date_arrivee DATETIME NOT NULL, places_totales INT NOT NULL, places_disponibles INT NOT NULL, #utilisateur_id INT REFERENCES utilisateurs(id), created_at DATETIME, updated_at DATETIME)

Légende

- Attribut souligné : clé primaire (PK).
- #attribut : clé étrangère (FK).

Contraintes d'intégrité

- `agence_depart_id` $\neq$ `agence_arrivee_id` : les agences de départ et d'arrivée doivent être différentes.
- `date_arrivee` $>$ `date_depart` : cohérence temporelle d'un trajet.
- `places_disponibles` $\leq$ `places_totales`.
- `places_totales` $>$ 0.
- `places_disponibles` $\geq$ 0.
- email UNIQUE dans la table utilisateurs.
- ville UNIQUE dans la table agences.

9. Procédure d'installation

Prérequis

- PHP $\geq$ 8.1 avec les extensions : `pdo_mysql`, `mbstring`, `xml`
- MySQL $\geq$ 5.7 ou MariaDB $\geq$ 10.3

- Composer (gestionnaire de dépendances PHP)
- Git

Étapes d'installation

1. Cloner le dépôt GitHub :

```
bash
git clone https://github.com/Maygae/touche-pas-au-klaxon.git
cd touche-pas-au-klaxon
```

2. Installer les dépendances PHP :

```
bash
composer install
```

3. Configurer l'environnement :

```
bash
cp .env.example .env
# Éditer le fichier .env avec vos paramètres de base de données
```

4. Créer la base de données :

```
bash
mysql -u root -p < sql/schema.sql
```

5. Alimenter la base de données avec le jeu d'essais :

```
bash
mysql -u root -p < sql/seed.sql
```

6. Lancer le serveur de développement :

```
bash
php -S localhost:8000 -t public
```


7. Accéder à l'application : Ouvrir <http://localhost:8000> dans un navigateur web.

10. Identifiants de connexion

Compte administrateur :

Champ	Valeur
Email	admin@klaxon.fr
Mot de passe	password
Rôle	admin

Compte utilisateur (exemple) :

Champ	Valeur
Email	alexandre.martin@email.fr
Mot de passe	password
Rôle	utilisateur

Remarque : tous les 20 utilisateurs du jeu d'essais partagent le même mot de passe par défaut : `password`

11. Technologies utilisées

Le projet met en œuvre plusieurs technologies choisies pour répondre aux contraintes du cahier des charges et aux besoins d'une application web moderne côté serveur.

Technologie	Version	Usage
PHP	≥ 8.1	Langage serveur, programmation orientée objet, MVC
MySQL / MariaDB	-	Base de données relationnelle
Bootstrap	5.3	Framework CSS responsive

Sass	-	Préprocesseur CSS pour organiser et factoriser les styles
Composer	-	Gestionnaire de dépendances PHP
izniburak/router	-	Routeur PHP pour la gestion des routes HTTP
vlucas/phpdotenv	5.x	Gestion des variables d'environnement (.env)
PHPUnit	-	Framework de tests unitaires PHP
PHPStan	1.x	Outil d'analyse statique pour la qualité du code
Bootstrap Icons	1.11	Bibliothèque d'icônes vectorielles

L'utilisation conjointe de ces outils permet d'obtenir une application structurée, testée et plus facilement maintenable : l'architecture MVC en PHP s'appuie sur Composer pour charger les dépendances, la base MySQL/MariaDB stocke les données métier, Bootstrap et Sass assurent un rendu responsive cohérent avec la charte imposée, et la qualité du code est renforcée par les tests PHPUnit et l'analyse statique PHPStan.

12. Mesures de sécurité

La sécurité constitue un point important du projet, notamment parce que l'application gère des comptes utilisateurs, des données personnelles de contact et des opérations d'écriture en base de données. Plusieurs mesures ont donc été mises en place afin de sécuriser l'authentification, les formulaires et les échanges avec la base de données

- **Hachage des mots de passe** : les mots de passe sont stockés de manière sécurisée grâce à la fonction `password_hash()` avec l'algorithme `bcrypt`.
- **Protection CSRF** : un jeton unique est généré et vérifié pour chaque formulaire afin de limiter les risques de falsification de requête.
- **Requêtes préparées PDO** : les accès à la base de données utilisent des requêtes préparées afin de prévenir les injections SQL.
- **Échappement HTML** : les données affichées sont traitées avec `htmlspecialchars()` pour réduire les risques de faille XSS.
- **Validation des entrées** : les données saisies par l'utilisateur sont contrôlées côté serveur, aussi bien sur le type que sur la cohérence métier.
- **Contrôle d'accès** : l'application distingue trois niveaux d'accès — visiteur, utilisateur et administrateur — gérés à l'aide de middleware.
- **Régénération de session** : la fonction `session_regenerate_id()` est utilisée lors de la connexion afin de limiter les risques liés au vol de session.

- **Variables d'environnement** : les identifiants de connexion à la base de données sont stockés dans un fichier `.env`, ce qui évite de les exposer directement dans le code source versionné.

13. Structure du code

L'application est organisée selon une architecture MVC (Modèle, Vue, Contrôleur), ce qui permet de séparer les responsabilités entre la gestion des données, la logique métier et l'affichage. Cette organisation rend le projet plus lisible, plus facile à maintenir et plus simple à faire évoluer.

Le chargement des classes est assuré par un autoload PSR-4 avec l'espace de noms `App\`. Ce choix facilite l'organisation du code et permet de réutiliser plus facilement les modèles, contrôleurs et vues dans d'autres projets proches, comme ceux mentionnés dans le brief (covoiturage, billetterie, achats groupés, cadeaux de Noël, etc.).

Les classes du cœur de l'application, notamment la connexion à la base de données, le contrôleur de base et la gestion des vues, ont été conçues de manière générique et documentées à l'aide de DocBlock. Cette documentation améliore la compréhension du code et permet à d'autres développeurs, ou à des stagiaires, de reprendre plus facilement le projet.

Enfin, la maintenabilité du code est renforcée par la présence de tests unitaires avec PHPUnit et par l'utilisation de PHPStan pour l'analyse statique. Ces outils facilitent la détection d'erreurs, la refactorisation et l'évolution de l'application dans le temps.

14. Bilan personnel, difficultés rencontrées et axes d'amélioration

Difficultés rencontrées

La principale difficulté rencontrée dans le cadre du projet Touche pas au Klaxon a été la gestion de l'avancement sur la durée. Contrairement à un exercice réalisé dans un temps très court, ce projet devait être mené jusqu'à une échéance définie, tout en étant réalisé en parallèle des cours et des autres travaux de formation. Cette contrainte m'a amené à structurer mon organisation, à planifier les tâches dans le temps et à prioriser les fonctionnalités essentielles afin de garantir une progression régulière.

Une autre difficulté a concerné l'architecture MVC, en particulier l'articulation entre la partie Core et les Models. Il a été nécessaire de bien comprendre le rôle des composants génériques du noyau de l'application (gestion de la connexion à la base de données, contrôleur de base, gestion des vues), puis de les intégrer correctement avec les modèles métiers. Ce travail m'a permis de construire une application cohérente, modulaire et maintenable.

La sécurité a également constitué un point de vigilance important tout au long du projet. Plusieurs mécanismes ont été mis en place, tels que le hachage des mots de passe, l'utilisation de requêtes préparées, la protection contre les attaques CSRF, l'échappement des données, la validation des entrées utilisateurs ainsi que la gestion des droits selon les rôles. L'intégration simultanée de ces éléments a nécessité une attention constante et une application rigoureuse des bonnes pratiques.

Enfin, la rédaction du fichier README.md a représenté une difficulté complémentaire. Il s'agissait de produire une documentation claire et exploitable, détaillant le fonctionnement de l'application, son installation et son utilisation, afin de permettre sa compréhension et sa reprise par un autre développeur dans de bonnes conditions.

Bilan personnel

Ce projet m'a permis de consolider ma compréhension du développement d'une application web complète, en articulant une interface utilisateur, une logique métier côté serveur et une base de données. Il m'a également offert l'opportunité de mettre en œuvre une architecture MVC dans un contexte concret, en intégrant des contraintes réelles de structuration du code, de sécurité et de documentation.

Sur le plan méthodologique, ce travail m'a amené à améliorer mon organisation personnelle. J'ai appris à découper un projet en tâches cohérentes, à prioriser les développements essentiels et à maintenir une progression régulière malgré un contexte de travail fractionné. L'utilisation d'un outil de versioning m'a également permis de mieux suivre l'évolution du projet et de sécuriser mon développement.

Enfin, ce projet m'a permis de prendre du recul sur le rôle du développeur. Au-delà de la production de fonctionnalités, il implique de garantir la qualité du code, sa lisibilité, sa maintenabilité, ainsi que la sécurité et la documentation de l'application. Ces éléments constituent des exigences essentielles dans un contexte professionnel.

Axes d'amélioration

Pour la suite, je souhaite approfondir ma maîtrise de l'architecture MVC, notamment en renforçant la séparation des responsabilités entre les différentes couches (Core, Models, Controllers, Views). Cet approfondissement me permettra de concevoir des applications plus modulaires, évolutives et conformes aux bonnes pratiques de développement.

Je souhaite également renforcer mes compétences en sécurité applicative, afin de systématiser l'intégration des mécanismes de protection dès la conception du projet et de mieux appréhender les enjeux liés à la sécurisation des applications web.

Enfin, je souhaite améliorer ma gestion de projet sur des développements menés dans la durée. Cela passe par une planification plus rigoureuse, un suivi régulier de l'avancement et une documentation plus structurée. L'objectif est de produire des projets plus aboutis, facilement maintenables et exploitables dans un contexte professionnel, notamment dans une logique de collaboration ou de reprise par d'autres développeurs.